Boutique App

System Architecture Guide — How Everything Connects, for New Learners

Prepared for: Alamgir | Folder: devops-ai-playbook / projects / boutique-microservices | Scope: Phase A (local Docker Compose)

This app is a small e-commerce site built the way real production systems often are: as several small, independent backend programs (called **microservices**) instead of one large program. This guide is a map of that system — what each piece is, how they talk to each other, and what actually happens on screen when you click a button. It's written for someone who has never seen this codebase before.

Three diagrams do most of the explaining: the overall map of every container, a walk-through of five things you actually do in the app, and a look at exactly how Docker wires ten separate containers into one working system. Read them in order — each one builds on the last.

Key Concepts, in Plain Language

Skip this section if these terms are already familiar. Everything after it assumes you know what these mean.

Container

A lightweight, isolated box that runs one program with its own filesystem and network address, but shares the host computer's operating system. Think of it as a much lighter version of a virtual machine. Every colored box in this guide's diagrams is one container.

Docker

The tool that builds and runs containers from a recipe file called a Dockerfile.

Docker Compose

A tool that starts many containers together as one system, using a single docker-compose.yml file to describe each container, its port, and how they can reach each other. Running `docker compose up` starts all ten containers in this app at once.

Microservices architecture

Splitting an application into several small, independently-runnable services (auth, products, orders, users...) instead of one big program. Each service can be built, deployed, and scaled on its own.

API Gateway

A single front door that all client requests go through, which then forwards (“proxies”) each request to whichever backend service actually handles it. In this app, that's the gateway container on port 3001.

REST API

A convention for structuring network requests around URLs and HTTP verbs, e.g. `GET /products` to read data, `POST /orders` to create something. Every service in this app exposes a REST API.

Database-per-service

Each microservice owns its own database and is the only thing allowed to read or write it directly. Other services that need that data have to ask over the network instead of querying the database directly. This app follows that rule: four logical databases, one per owning service.

SPA (Single-Page Application)

A web app, like this one's frontend, that loads one HTML page and then uses JavaScript to swap content in and out as you navigate, instead of reloading the whole page from the server each time.

nginx

A fast, general-purpose web server. Here it does one job: serve the React app's pre-built HTML/CSS/JS files to your browser.

Reverse proxy

A server that sits in front of other servers and forwards requests to them on the client's behalf. Both nginx (for static files) and the gateway (for API calls) act as reverse proxies in this app.

Token-based auth

Instead of sending a password with every request, the user logs in once, gets back a token, and sends that token on every later request to prove who they are. This app implements a simplified version of that idea —

see the note in the last section.

1. The Big Picture

Ten containers, started together by one docker-compose.yml file. The browser is the only piece that isn't a container — it's your computer's normal web browser, running the React app that was sent to it.

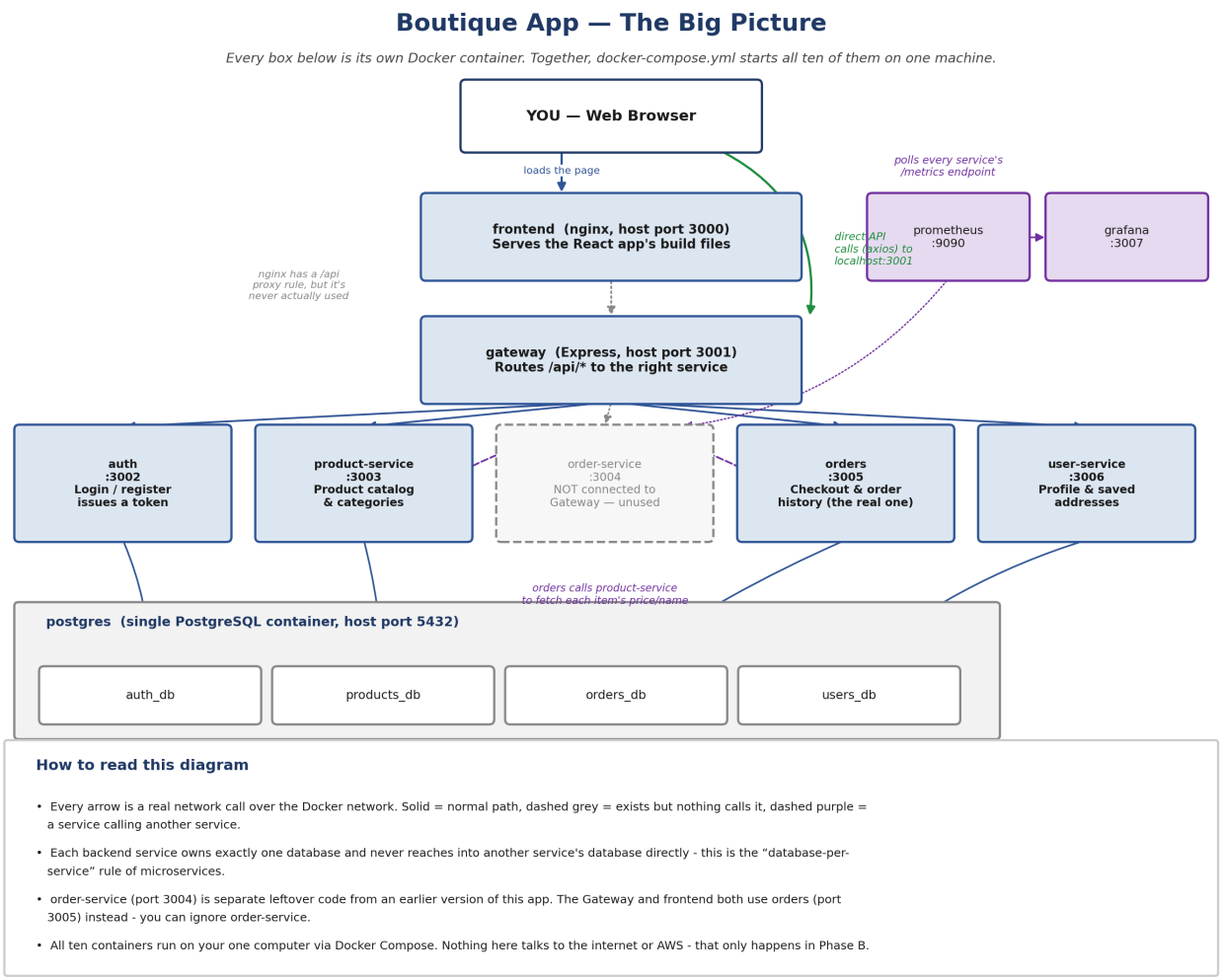

Figure 1 — Every container in the system, what it does, and what it's allowed to talk to.

2. Service Reference Table

The same information as Figure 1, in lookup form.

Service	Port	Tech	Database	What it actually does
frontend	3000	React + nginx	—	Serves the built React app to your browser
gateway	3001	Express	—	Routes /api/auth, /api/products, /api/orders, /api/users to the right service
auth	3002	Express + bcryptjs	auth_db	Register, login, logout — issues a simple token
product-service	3003	Express + sharp	products_db	Product catalog, product detail, categories
order-service	3004	Express (plain JS)	orders_db (own schema)	Leftover/unused code — not called by the Gateway or frontend
orders	3005	Express + Joi	orders_db	Checkout and order history — the one the app actually uses
user-service	3006	Express + Joi	users_db	Profile and saved addresses
postgres	5432	PostgreSQL 15	all 4 DBs	One database server hosting four separate logical databases
prometheus	9090	Prometheus	—	Collects metrics from every service's /metrics endpoint
grafana	3007	Grafana	—	Dashboards built from Prometheus's data

3. How a Request Actually Travels

Architecture diagrams show what's possible; this shows what actually happens for five things you do as a shopper. Notice step 3 — not everything in a web app touches the backend at all.

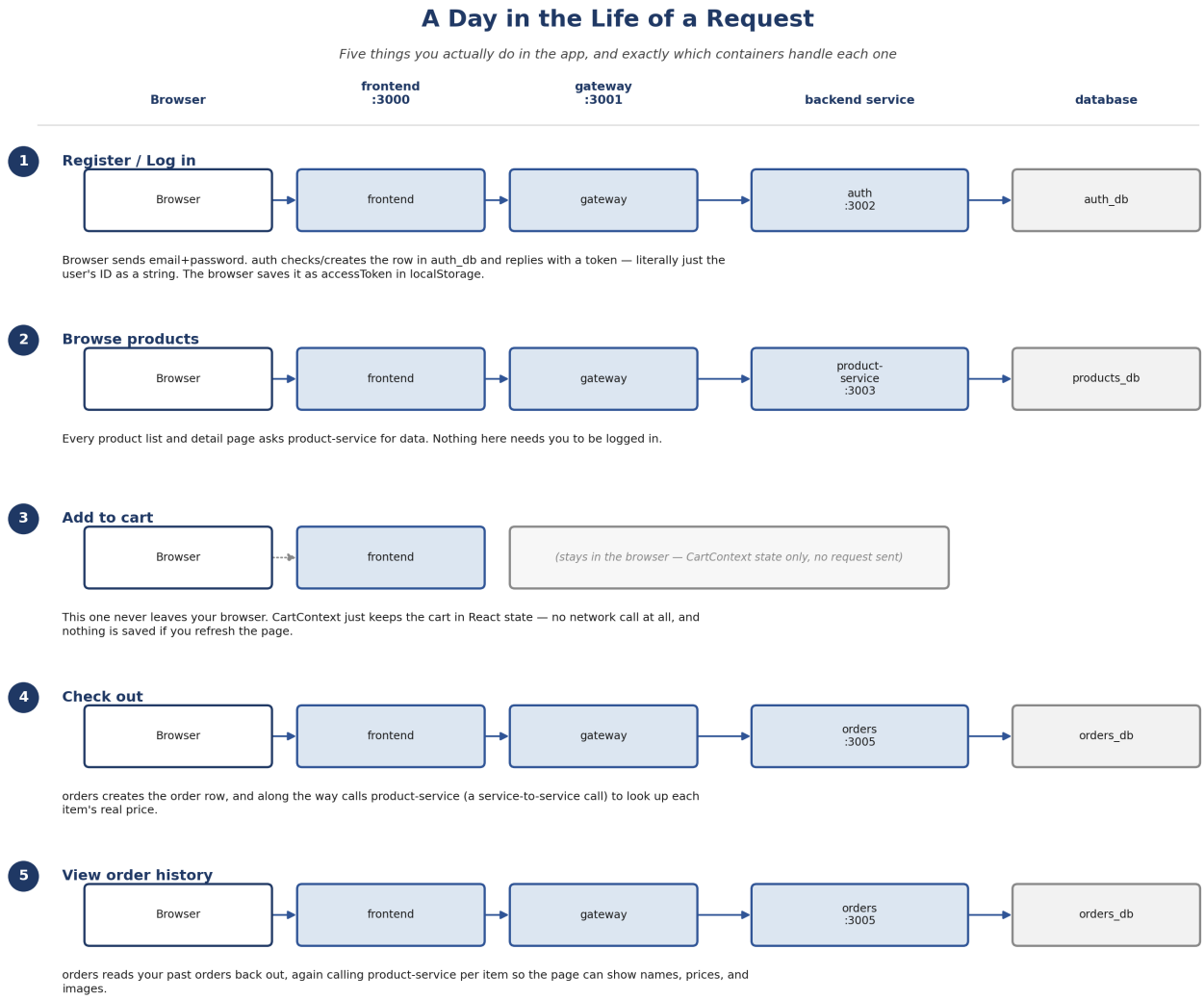

Figure 2 — Five real user actions, traced through every container involved.

4. How Docker Compose Wires It All Together

docker-compose.yml gives every container a name and a private network to talk on. Containers reach each other by that name (e.g. `http://product-service:3003`) — that hostname only resolves inside Docker, never from your browser. Your browser instead reaches containers through the host ports Compose maps to your machine, listed on the left of each box below.

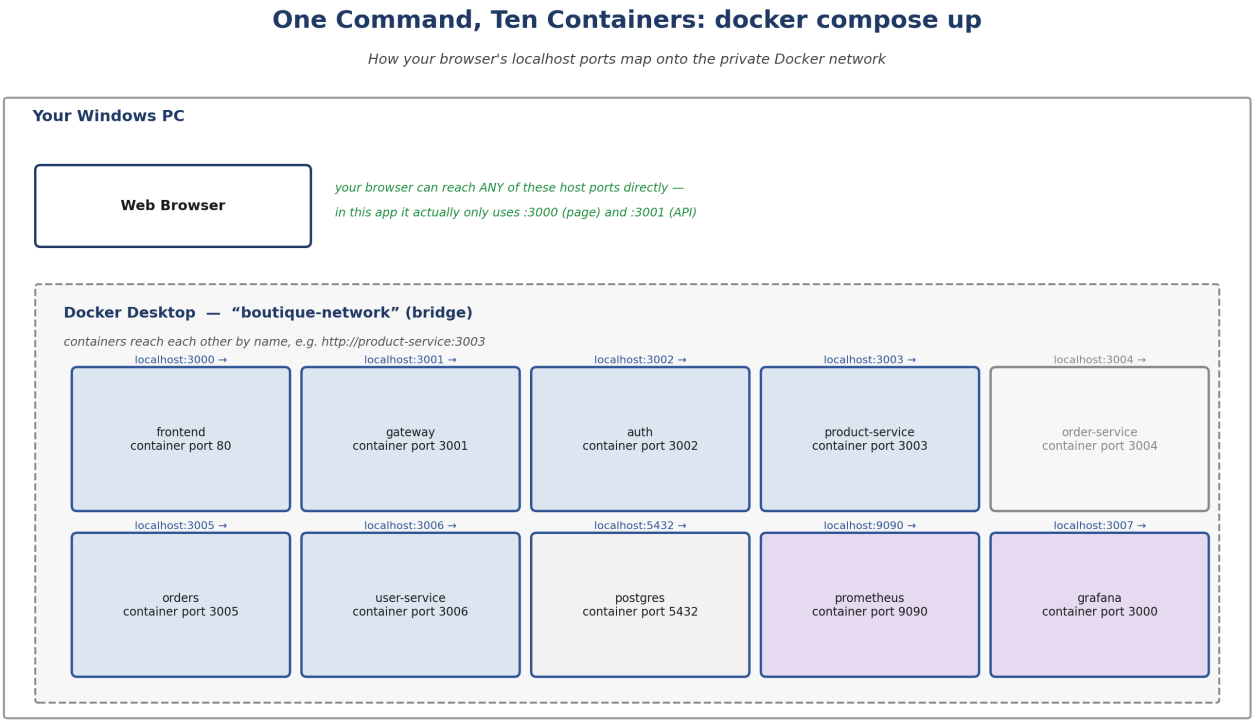

Figure 3 — The private Docker network, and which host ports reach into it.

> **Where the 4 databases actually come from:** database/init/10-create-databases.sh runs first and literally executes CREATE DATABASE four times (auth_db, products_db, orders_db, users_db) inside the one postgres container. Only after those databases exist does database/init/20-init-schema.sql run and create tables inside each one.

5. Things That Look Like Bugs (But Aren't) — and One That Was

Real codebases, including this sample one, accumulate leftover code and shortcuts. Knowing which oddities are intentional simplifications and which are actual mistakes will save you time.

The “token” isn't a real JWT

auth's login and register routes return `token: user.id.toString()` — the token is just the user's database ID as plain text, not a cryptographically signed JSON Web Token (JWT). It works fine for this demo because nothing here checks a signature, but it would not be safe in a real product: anyone could forge a token just by guessing or copying a UUID. Real systems sign tokens with a secret key so they can't be faked.

order-service is dead code

There are two similarly-named services: order-service (port 3004) and orders (port 3005). Only orders is wired into the Gateway's routing table and only orders is what Checkout and Order History actually call. order-service still runs as a container and even has its own tiny schema, but nothing in the running app ever sends it a request — it's safe to ignore.

nginx's /api proxy rule is configured, but unused

frontend/nginx.conf includes a rule to forward `/api/` requests to the gateway container. In practice the React app never calls a relative `/api/...` path — its API client is hardcoded to call `http://localhost:3001/api` directly (see frontend/src/services/api.ts), bypassing nginx entirely. Both paths would work; only one is actually used.

A real bug: the gateway couldn't reach user-service

gateway/src/index.ts reads `USERS_SERVICE_URL` (plural) to know where to send `/api/users` requests, but docker-compose.yml was setting `USER_SERVICE_URL` (singular) for the gateway container — a naming mismatch. The gateway silently fell back to its own default, which pointed at the wrong port entirely. This has been corrected directly in docker-compose.yml so Profile and address requests now actually reach user-service. Run `docker compose up -d --force-recreate gateway` once to pick up the fix.

> **Why this matters:** None of this is unusual. Every real codebase has some leftover code, a config value that quietly stopped being read, or a naming mismatch nobody noticed. Learning to spot the difference between “intentional simplification” and “actual bug” by reading the source, instead of assuming, is one of the most useful habits you can build as a developer.